Inhaltsverzeichnis

1. Einleitung **1**
1.1. Hintergrund und Motivation . . . 1
1.2. Problemstellung . . . 1
1.3. Abgrenzung . . . 2

2. Methodik **2**
2.1. Benchmark Aufbau . . . 2
2.2. Datensatz . . . 3
2.2.1. Ausschlusskriterien . . . 3
2.2.2. Vorverarbeitung . . . 4
2.3. Evaluierung . . . 4
2.3.1. Äquivalenz des Outputs . . . 4
2.3.2. Statische Code Metriken . . . 5
2.3.3. Weitere Metriken . . . 6
2.3.4. Aggregation . . . 7
2.4. Die Benchmark Benutzen . . . 8

3. Ergebnisse **8**
3.1. Tests . . . 8
3.1.1. Auswertung der Übersetzer . . . 9

4. Diskussion **11**
4.1. Die Benchmark . . . 11
4.2. Limitationen . . . 11

5. Fazit und Ausblick **11**
5.1. Weiterführende Arbeit . . . 11
5.2. Zusammenfassung . . . 12

A. Prompts **15**
A.1. System Prompt . . . 15
A.2. Base Prompt . . . 15
A.3. AST Prompt . . . 15

1. Einleitung

1.1. Hintergrund und Motivation

COmmon Business-Oriented Language (COBOL) ist eine Programmiersprache, welche 1959 mit dem Ziel einer universell einsetzbaren Programmiersprache entwickelt wurde. Sie sollte auch für nicht Programmierer leicht verständlich sein und ist deshalb stark an der englischen Sprache angelehnt. Seither wurde COBOL in vielen Branchen eingesetzt und auch heute noch basiert ein Grossteil der Finanzinfrastruktur darauf, dies ist vor allem seiner Robustheit und Zuverlässigkeit zuzuschreiben (TecnoDigital, 2025).
Seit 1959 hat es viele technologische Weiterentwicklungen gegeben; neue Programmiersprachen, Programmierparadigmen, Cloud Computing etc. Viele der ursprünglich in COBOL entwickelten Systeme funktionieren aber auch heute noch gut und zuverlässig, aus einer Business Perspektive gibt es deswegen zumindest kurzfristig wenige Anreize dazu, die bestehenden Systeme in einer moderneren Sprache neu zu schreiben (TecnoDigital, 2025).

Heutzutage bevorzugen die meisten Entwickler aber modernere Programmiersprachen wie Java, C# etc. Viele COBOL Entwickler gehen in den Ruhestand und niemand ist da um diese Lücken zu schliessen. Ohne Entwickler kann die Software nicht weiterentwickelt, gewartet und repariert werden. Bedingt dadurch müssen Unternehmen, welche COBOL verwenden diesen Code in eine neuere Programmiersprache übersetzen, um langfristig bestehen zu können.

Da dies eine grosse Aufgabe ist, wird versucht diesen Prozess zu vereinfachen. Mit dem Aufkommen von Large Language Machines (LLMs) ergibt sich eine Möglichkeit wie das bewerkstelligt werden könnte. Momentan zeigt ein naiver Ansatz mit LLMs aber noch, dass diese noch nicht gut genug für eine zuverlässige Übersetzung sind. Grund dafür ist unter anderem der Mangel an frei zugänglichem COBOL Code zum Training von LLMs.

1.2. Problemstellung

Large Language Machines (LLMs) können dabei helfen den COBOL Code automatisiert zu übersetzen, mit ihrer Hilfe kann die Übersetzung vereinfacht oder in einigen Fällen sogar erst ermöglicht werden. Dadurch, dass verglichen mit anderen Programmiersprachen die Menge an frei zugänglichem COBOL Code sehr klein ist haben die meisten LLMs aber nur eine begrenztes Verständnis der Sprache. Um eine Zuverlässige Übersetzung gewährleisten zu können müssen deshalb kreative Ansätze eingesetzt werden, um die Übersetzungsqualität zu gewährleisten.

Um die Übersetzungsqualität zu verbessern ist es deswegen wichtig, dass unterschiedliche Ansätze objektiv miteinander verglichen werden können. Ziel dieser Arbeit ist deshalb die Entwicklung einer Benchmark für die Übersetzung von COBOL zu Java.

Die Anforderung an einen Übersetzer ist, dass das resultierende Programm Äquivalent zum COBOL Programm verhält, das bedeutet, dass bei gleichen Inputs auch wieder die exakt gleichen Outputs produziert werden. Dies ist nötig, da COBOL Programme meist in Umgebungen laufen, wo kein anderes Verhalten geduldet wird. Zusätzlich soll aber auch die Qualität des resultierenden Java Codes beurteilt werden. Wenn der resultierende Code von schlechter Qualität ist, wurde im Sinne der Wartbarkeit nicht viel gewonnen.

1.3. Abgrenzung

Das Projekt fokussiert sich auf die Entwicklung einer Benchmark, nicht auf die Entwicklung von Übersetzungsansätzen.
In der Literatur konnten noch einige andere Benchmarks identifiziert werden, keiner davon testet die Übersetzung mittels Äquivalenztests und statischer Code Analyse.

CodeTransOcean (Yan et al., 2023) testet die Übersetzung von Nischensprachen zu Weiter verbreiteten Sprachen mittels eines Datensets an Programmen, welche parallel in mehreren Programmiersprachen vorliegen. Die Übersetzung wird dann unter anderem mittels BLEU mit der Referenzimplementierung verglichen.

MainframeBench (Dau et al., 2024) Testet LLMs auf Mainframe Wissen, COBOL multiple choice, Q&A und COBOL Code Zusammenfassen.

OpenCBS: COBOL Defects Benchmark (Lee et al., 2022) Stellt eine Benchmark Suite zusammen mit Defektem COBOL Code, welcher aus Entwickler Foren extrahiert wurde. Ziel davon ist das Entwickeln und Testen von Entwickler-Tools.

Andere Ansätze zur Unterstützung der COBOL Übersetzung / Mainframe Modernisierung umfassen: Extrahieren von Business Rules (Cosentino et al., 2013; Shah et al., 2025), Erklärungen und Zusammenfassung von COBOL Code (Dau et al., 2024; Lei et al., 2025)

Proprietäre Tools: Vor allem von IBM wurden schon einige Tools (IBM, 2024) zur Übersetzung und deren Validierung (Kumar et al., 2024) gebaut. Diese sind aber nicht Open Source. Sie zeigen aber, das mit genug Unterstützung die automatisierte Übersetzung durchaus Möglich ist.

2. Methodik

Für das erstellen der Benchmark wird als erstes ein Datensatz an COBOL Dateien zusammengestellt. Anschliessend werden von den COBOL Programmen die Outputs aufgezeichnet. Für die dafür nötigen Input Dateien müssen Beispieldaten generiert werden. Wenn alle Testcases so vorbereitete sind, werden die COBOL Programme mit dem Übersetzer nach Java übersetzt und anschliessend die Übersetzung bewertet.

2.1. Benchmark Aufbau

Die Idee der Benchmark ist, dass wenn wir ein COBOL Programm nach Java übersetzen, das Programm mit den gleichen Inputs die gleichen Outputs produzieren sollte, wie das Original COBOL Programm. Um das testen zu können werden für jedes COBOL Programm alle benötigten Input-Dateien und deren Inhalt durch ein LLM (Claude Sonnet 4.5 (Anthropic, 2025)) generiert. Danach wird das COBOL Programm ausgeführt und die produzierten Dateien und Terminal Outputs eingefangen und als Testcase gespeichert. Dies wird für jedes Programm wiederholt.
Um einen Übersetzer zu testen kann nun jeder Testcase einfach in einem Ordner mit den benötigten Dateien aufgebaut werden. Anschliessend kann dort das generierte Java Programm kompiliert und ausgeführt werden. Die vom Java Programm generierten Outputs

werden mit den vom COBOL Programm generierten verglichen und auf ihr Ähnlichkeit bewertet. Zusätzlich wird auch die Code Qualität des Java Programmes bewertet.
Der Ablauf ist in Algorithmus 1 nochmals übersichtlich dargestellt.

Algorithm 1 Benchmark Ablauf

```
User stellt einen Übersetzer bereit, welcher das BaseCob2JavaConverter Interface implementiert.
for Testcase t im Benchmark do
    Erstelle Ordner für t mit allen nötigen Dateien
    Übergebe COBOL-Code von t an Übersetzer
    Erzeuge Java-Code
    Kompiliere Java-Code
    Führe das Java Programm aus
    Fange alle Outputs ein
    Vergleiche produzierte Outputs mit erwartetem Output
    Generiere Ähnlichkeitsmetriken für den produzierten Output
    Generiere statische Code-Metriken für den produzierten Code
end for
Aggregiere die Metriken aller Testcases
```

2.2. Datensatz

Die Verwendeten COBOL Dateien stammen aus den TheStack (Kocetkov et al., 2022) und CodeTransOcean-NicheTrans (Yan et al., 2023) Datensätzen. TheStack ist ein Datensatz, welcher für das Pre-Training von Code LLMs zusammengestellt wurde, er enthält Code in 358 Programmiersprachen welcher aus GitHub Repositories gesammelt wurde, ein kleiner Anteil davon COBOL. CodeTransOcean-NicheTrans enthält Implementierungen des selben Codes in mehreren Programmiersprachen. Die COBOL Dateien wurden aus beiden Datensätzen extrahiert und gesammelt.

2.2.1. Ausschlusskriterien

Die Dateien wurden vor der weiteren Verarbeitung gefiltert nach folgenden Kriterien:

1. **Kompilierbarkeit**: Alle Programme, welche mit dem GNUCobol Compiler (Free Software Foundation, 2023) nicht erfolgreich Kompiliert werden können, werden ausgeschlossen.

2. **Länge der Dateien**: Programme kürzer als 60 Linien Code werden ausgeschlossen. So werden sehr einfach Codes ausgeschlossen.

3. **Ähnliche Dateien**: Duplikate und fast Duplikate werden ausgeschlossen

4. **Command Line Argumente**: Programme, welche Command Line Argumente benutzen werden ausgeschlossen.

5. **Interaktive Programme**: Programme bei denen ein User Interaktiv eingaben machen muss werden ausgeschlossen.

Nach dem Anwenden der Ausschlusskriterien sind noch 318 COBOL Programme übrig.

2.2.2. Vorverarbeitung

In den übrig gebliebenen Dateien werden alle Dateipfade so umgeschrieben, das sie relativ zur COBOL Datei im gleichen Verzeichnis liegen. Dies damit alle Dateien, welche vom Programm benötigt werden im gleichen Ordner liegen und nach dem Ausführen des Programmes alles einfach wieder aufgeräumt werden kann.

2.3. Evaluierung

Um die Güte eines Übersetzers zu bewerten werden die in Tabellen 1 2 3 definierten Metriken betrachtet.

2.3.1. Äquivalenz des Outputs

Um die Äquivalenz des Outputs zu testen werden jeweils alle generierten Outputs $\hat{y}_t$ (Dateien, Stdout, etc.) eingefangen und miteinander verglichen. Dabei wird jeder erwartete Output $y_{t,i}$ mit dem produzierten $\hat{y}_{t,i}$ verglichen und die Ähnlichkeits-Metrik berechnet. Anschliessend wird über alle Outputs $(y_{t,i}, \hat{y}_{t,i}), i \in \{1, \ldots, n_t\}$ ein Weighted Average gebildet, wobei die Länge der erwarteten Outputs die Weights vorgeben. Folgend werden die Formeln jeweils generell, für ein Output Tupel $(y, \hat{y}) \equiv (y_{t,i}, \hat{y}_{t,i})$ definiert.

Metrik	**Beschreibung**
Exact Match	Überprüft, ob der vom Java-Programm erzeugte Output exakt mit dem COBOL-Output übereinstimmt. Exact Match Ratio: $$\text{EMR} = \frac{1}{T}\sum_{t=1}^{T} \mathbb{1}\{y_t = \hat{y}_t\}$$
Cosine Similarity	Mass für die Ähnlichkeit zwischen zwei Vektoren. Wird zwischen den Embeddings von $y, \hat{y}$ berechnet. Gibt den Cosinus des Winkels zwischen den Vektoren an (Levy et al., 2024). $$sim_{Cos}(y, \hat{y}) = \frac{\langle y \cdot \hat{y} \rangle}{\lVert y \rVert \, \lVert \hat{y} \rVert}.$$
Levenshtein Similarity	Minimale Anzahl einfügenden, löschenden oder ersetzenden Operationen, um Zeichenkette y in Zeichenkette $\hat{y}$ umzuwandeln ("Levenshtein-Distanz", 2025). Ihre normalisierte Version (Wu & Martin, 2024): $$sim_{levenshtein} = 1 - \left(\frac{\text{Levenshtein Distance}}{\max\left(\text{length}(y), \text{length}(\hat{y})\right)}\right)$$
Jaccard-Index (Intersect over Union)	Berechnet die Ähnlichkeit von zwei Sets ("Jaccard-Koeffizient", 2025). In diesem Fall bestehen die Sets aus Wörtern. $$J(y, \hat{y}) = \frac{\lvert y \cap \hat{y} \rvert}{\lvert y \cup \hat{y} \rvert}$$

Tabelle 1: Evaluationsmetriken zur Output-Ähnlichkeit

- y_t: Erwarteter Output für Testcase t
- $\hat{y}_t$: Produzierter Output für Testcase t
- $y_{t,i}$: Erwarteter Output $i \in \{1, \ldots, n_t\}$ für Testcase $t \in \{1, 2, \ldots, T\}$
- n_t: Anzahl Erwarteter Outputs für Testcase t
- $\mathbb{1}\{\}$: Indikatorfunktion für das eintreffen eines Events. $\mathbb{1}\{E\} = 1$, wenn E eintritt, sonst 0

2.3.2. Statische Code Metriken

Zusätzliche zu den Ähnlichkeitsmetriken wird der generierte Java Code auch auf die Code Qualität Bewertet. Dies wird mittels einigen Code Qualität Metriken gemacht, welche in Tabelle 2 aufgeführt sind. Der Maintainability index ist dabei eine Zusammenfassung für die Code Qualität und wird so auch von Visual Studio verwendet, um die Code Qualität zu bewerten.

Metrik	**Beschreibung**
Halstead Volume	Misst die Grösse des Programmes $$V = N \cdot \log_2(n)$$ (Shivani Virmani, 2025)
Halstead Effort	Aufwand zum Verstehen/Schreiben des Programmes $$E = V \cdot D$$ (Shivani Virmani, 2025)
Cyclomatic Complexity	Metrik für die Komplexität eines Programms, gemessen an der Anzahl Wege durch das Programm an der Anzahl Nodes n, Kanten e und p verbundenen Komponenten des Kontroll Fluss Graphen eines Programmes. $$\mathrm{v}(G) = \mathrm{e} - \mathrm{n} + 2\mathrm{p}$$ (akash1295, 2025; McCabe, 1976)
Lines Of Code	Die Anzahl Zeilen des Programmes
Maintainability Index	Metrik, welche verwendet wird, um die Wartbarkeit eines Programmes zu bewerten. Wird auch von Microsoft verwendet. $$\begin{aligned} MI = &\max(0, (171 - 5.2 \cdot \ln(\text{Halstead Volume}) \\ &- 0.23 \cdot \text{Cyclomatic Complexity} \\ &- 16.2 \cdot \ln(\text{Lines of Code})) * 100/171) \end{aligned}$$ Werte von 0-9 bedeuten tiefe Wartbarkeit, 10-19: Mittlere Wartbarkeit, 20-100: Gute Wartbarkeit (Mikejo5000, 2025).

Tabelle 2: Evaluationsmetriken zur Code Qualität

Cyclomatik Complexity McCabe (1976) schlägt Cyclomatic Complexity > 10 als einen Indikator vor, dass Funktionen zu komplex sind und potentiell refactored werden müssen.

Halstead Software Metrics sind Metriken, um die Komplexität von Software zu bewerten, basierend auf der Anzahl Operatoren und Operanden in einem Programm. Operatoren (+,-, etc.) führen Aktionen auf Operanden (Strings, Nummern etc.) aus.

- $n1$: Grösse des Sets der unterschiedlichen Operatoren
- $n2$: Grösse des Sets der unterschiedlichen Operanden
- $N1$: Anzahl der Operatoren
- $N2$: Anzahl der Operanden
- $n = n1 + n2$: Halstead Vokabular
- $N = N1 + N2$: Halstead Programm Länge
- $D = \frac{n1}{2} \cdot \left(\frac{N2}{n2}\right)$ Halstead Difficulty

(Shivani Virmani, 2025)

2.3.3. Weitere Metriken

Zusätzlich zu den in Tabelle 1 und 2 aufgeführten Metriken werden in Tabelle 3 auch noch Statistiken über den Übersetzungsprozess gesammelt um beurteilen zu können, wie Zuverlässig die Übersetzungsmethode ist, wie schnell etc.

Metrik	**Beschreibung**
Generation Success	Anteil der Testcases, wo der Übersetzungsprozess ohne Fehler abgelaufen ist. Generation Success Rate (GSR): $$\mathrm{GSR} = \frac{1}{\lvert T \rvert} \sum_{t=1}^{T} \mathbb{1}\left\{\mathrm{GenerationSuccessful}\left(t\right)\right\}$$
Compilation Success	Anteil der Testcases, wo das generierte Programm ohne Fehler Kompiliert. Compilation Success Rate (CSR): $$\mathrm{CSR} = \frac{1}{T} \sum_{t=1}^{T} \mathbb{1}\left\{\mathrm{CompilationSuccessful}\left(\hat{y}_t\right)\right\}$$
Run Success	Anteil der Testcases, wo sich das generierte Programm ohne Fehler ausführen Lässt. Run Success Rate (RSR): $$\mathrm{RSR} = \frac{1}{T} \sum_{t=1}^{T} \mathbb{1}\left\{\mathrm{RunSuccessful}\left(\hat{y}_t\right)\right\}$$
Conversion Time	Durchschnittliche Zeit zum übersetzen eines Programmes $$\mathrm{MeanCT} = \frac{1}{T} \sum_{t=1}^{T} \mathbb{1}\left\{\mathrm{ConversionTime}\left(t\right)\right\}$$
Total Conversion Time	Total benötigte Zeit zum Übersetzen aller Programme in der Benchmark $$\mathrm{TotalCT} = \sum_{t=1}^{T} \mathbb{1}\left\{\mathrm{ConversionTime}\left(t\right)\right\}$$

Tabelle 3: Evaluationsmetriken zum Übersetzungsprozess

2.3.4. Aggregation

Die Metriken werden über alle Testcases Aggregiert

Boolean Metriken, deren Werte 1 oder 0 annehmen werden mit einer Summe und einer Ratio aggregiert.

$$\mathrm{metric}_{\mathrm{sum}} = \sum_{t=1}^{T} \mathrm{metric}_t$$

$$\mathrm{metric}_{\mathrm{ratio}} = \frac{1}{T} \sum_{t=1}^{T} \mathrm{metric}_t$$

Numerische Metriken werden mit mean, min, max, σ aggregiert

$$\text{metric}_{\text{mean}} = \frac{1}{T}\sum_{t=1}^{T}\text{metric}_t$$

$$\text{metric}_{\text{std}} = \sqrt{\frac{1}{T}\sum_{t=1}^{T}(\text{metric}_t - \text{metric}_{\text{mean}})^2}$$

$$\text{metric}_{\text{min}} = \min_{t\in[1,\ldots,T]}\text{metric}_t$$

$$\text{metric}_{\text{max}} = \max_{t\in[1,\ldots,T]}\text{metric}_t$$

2.4. Die Benchmark Benutzen

Um einen Übersetzer mit der Benchmark testen zu können, muss in Python eine Klasse erstellt werden, welche ein `BaseCob2JavaConverter` Interface implementieren.
Ein `BaseCob2JavaConverter` muss eine `convert(cobol_code: str) -> str` Methode implementieren, welche den COBOL Source Code als String akzeptiert und den generierten Java Code als String zurückgibt.

Eine Instanz des neuen Übersetzers muss in der main Methode in `run_benchmark.py` erstellt werden und dem Benchmark Objekt übergeben werden. In dieser Datei können für den Übersetzer auch noch einige Metadaten angegeben werden.

Ausserdem ist es auch möglich (z.B. wegen eines Fehlers) einen abgebrochenen Benchmark Durchlauf fortzusetzen, indem `resume_previous_run = True` gesetzt wird und der Ordner angegeben wird, wo die Übersetzen Programme abgelegt werden.

Wenn die Benchmark fertig gelaufen ist, sind die Übersetzen Programme im `conversions` Ordner zu finden, alle noch mit den generierten Output-Dateien, COBOL Code und Java Code. Im `benchmark_results` Ordner wird ein JSON Dokument erstellt, welches die Aggregierten Metriken enthält.
Im `conversions` Ordner werden während der Benchmark Ausführung nach jedem Testcase die aktuellen aggregierten Metriken abgelegt, sowie auch die Metriken pro Testcase.

3. Ergebnisse

3.1. Tests

Zur Veranschaulichung der Benchmark und Inspiration wurden zwei Übersetzer-Typen mit drei unterschiedlichen LLMs getestet und ausgewertet.

Base Der erste Übersetzer-Typ ist ein einfaches LLM, mit den Anweisungen den COBOL Code nach Java zu übersetzen.

Für den Base-Typ werden die LLMs Claude-Sonnet-4.5, Claude-Haiku-4.5 und Qwen3-Coder-30B-A3B-Instruct getestet. (Anthropic, 2025; Yang et al., 2025) Das Qwen-3 wird auf einer Nvidia A100 betrieben.
Der User Prompt des Base Typs ist im Anhang A.2 und der System Prompt im Anhang A.1.

AST Der Zweite getestete Übersetzer-Typ ist wieder LLM basiert, aber zusätzlich zum COBOL Code wird dem LLM auch noch der Abstract Syntax Tree (AST) des COBOL Programmes mitgegeben. Ein AST stellt den Programmablauf in einer Baum Datenstruktur dar. Er enthält nicht jedes Detail des Programmes, Abstrahiert Sie oder stellt sie implizit durch die Struktur dar. Er fokussiert sich auf die Strukturellen Elemente des Programmes. ("Abstract syntax tree", 2025)

Für den AST-Typ werden die LLMs Claude-Haiku-4.5 und Qwen3-Coder-30B-A3B-Instruct getestet. Claude-Sonnet-4.5 wird aus Kostengründen hier nicht getestet. Der AST wird ziemlich gross und benötigt viele Input-Tokens.
Der User Prompt des AST Typs ist im Anhang A.2 und der System Prompt im Anhang A.1.

3.1.1. Auswertung der Übersetzer

In Abbildung 1 wurden die wichtigsten Ergebnisse der Benchmark für die fünf getesteten Übersetzer visualisiert.

In **a)** ist zu erkennen, dass die Rate zum Erfolgreichen Ausführen des generierten Codes bei Base-Sonnet-4.5 am höchsten ist, bei ihm ist der Unterschied zwischen Kompilier-Rate und Ausführbar-Rate am kleinsten, was zeigt dass die Rate an Runtime Fehlern am geringsten ist. Überraschenderweise sind die lokalen Qwen-3 basierenden Übersetzer erfolgreicher als die Haiku-4.5 Übersetzer, zumindest beim generieren und Kompilieren, beim Ausführen hingegen sind sie fast gleich gut. Lokale LLMs sind also nicht zu vernachlässigen.

In **b)** kann die Ähnlichkeit des generierten Outputs anhand der Cosine, Jaccard und Levenshtein Metriken verglichen werden. Hier zeigt sich bei allen Übersetzern ein ähnliches Muster: Levenstein am höchsten, Cosine Similarity als zweites, gefolgt vom Jaccard.
Auch hier erzielt Base-Sonnet-4.5 die besten Ergebnisse. Gefolgt wird es von beiden Haiku-4.5 Übersetzern, wobei das mit AST überraschenderweise tiefer ist. Es wird vermutet, dass Haiku-4.5 nicht so gut mit dem sehr langen Input Text das AST's zurecht kommt und dass es gar keinen Nutzen daraus ziehen kann. Jedoch ist es bei Qwen genau anderst herum, dort ist das Base Modell leicht schlechter. Qwen-3 hat ein Limit von 256K Input Tokens, und Haiku-4.5 200K Input Tokens. Evtl kann deshalb Qwen-3 mit den langen Inputs besser umgehen.

In **c)** wird die Code Qualität anhand des Maintainability Index verglichen.
Alle Übersetzer schaffen es im Durchschnitt Code von "High Maintainabilityßu erzeugen. Überraschenderweise erzielt Qwen-3 hier die besten Werte, wahrscheinlich weil es das einzige Model ist, welches für speziell auf Code ausgerichtet ist.

In **d)** ist ersichtlich, dass das Base-Haiku-4.5 mit am schnellsten ist, gefolgt vom Base-Sonnet-4.5. Mit Abstand am Längsten braucht AST-Qwen-3,
In Tabelle 4 sind alle Metriken detailliert aufgeführt. Die fett gedruckten Werte markieren den besten Wert pro Metrik.
Es ist klar erkennbar, dass Base-Sonnet-4.5 am besten funktioniert. Jedoch ist das Base-Haiku-4.5 in den Ähnlichkeitsmetriken nur wenig schlechter, ist aber erheblich billiger, was bei grossen Projekten durchaus eine Rolle spielt. Auch ersichtlich ist, dass aber Sonnet-4.5

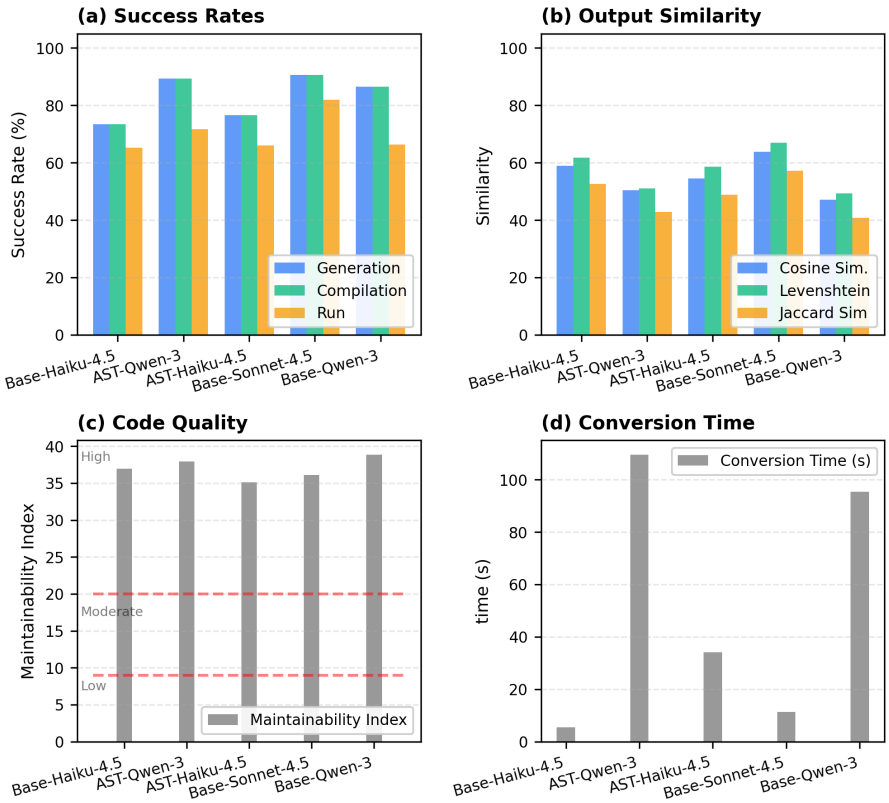

Abbildung 1: Vergleich der getesteten Übersetzer

	AST		BASE		
	Haiku-4.5	**Qwen-3**	**Haiku-4.5**	**Qwen-3**	**Sonnet-4.5**
Genenration Success	77.0	89.0	73.0	86.0	**91.0**
Compilation Success	77.0	89.0	73.0	86.0	**91.0**
Run Success	66.0	72.0	65.0	66.0	**82.0**
Cosine Similarity	0.54	0.5	0.59	0.47	**0.64**
Levenshtein Similarity	0.59	0.51	0.62	0.49	**0.67**
Jaccard Similarity	0.49	0.43	0.53	0.41	**0.57**
Halstead Volume	1760.99	1467.04	1522.64	**1393.74**	1674.37
Cyclomatic Complexity	15.81	12.94	**14.45**	12.96	15.1
Lines of Code	82.37	69.5	73.59	**65.25**	76.73
Maintainability Index	35.13	37.97	36.96	**38.86**	36.1
Conversion Time (s)	34.08	109.57	**5.39**	95.4	11.26

Tabelle 4: Metriken pro Übersetzer

dazu neigt längere und Komplexere Programme zu schreiben, als die anderen Übersetzer.

Um generell sagen zu können, ob sich die Verwendung eines AST im Prompt lohnt, müssten noch mehr Modelle ausprobiert werden, da AST-Haiku-4.5 und AST-Qwen-3 gegensätzliche Trends zeigen, Qwen-3 besser mit, Haiku-4.5 besser ohne AST.

4. Diskussion

4.1. Die Benchmark

Die Benchmark bietet eine Objektive Grundlage zum vergleichen unterschiedlicher Ansätze zur Übersetzung. Dank der Verwendung einer Vielzahl von diversen Metriken können die Übersetzer genau miteinander verglichen und Stärken und Schwächen identifiziert werden. Es gibt Forschenden einen guten Anhaltspunkt, um beurteilen zu können, ob eine Technik funktioniert und weiterverfolgt werden sollte.

4.2. Limitationen

Die Benchmark ist ein guter Start, jedoch gibt es einige Limitationen.

- Es wurde nur frei zugänglicher Code verwendet. Dieser Code ist nicht unbedingt Repräsentativ für Code, wie er bei Banken, Versicherungen etc. eingesetzt wird. Viele Beispiele aus dem Datensatz sind z.B. von Coding Challenges. Also nicht CODE, wie er in einem Produktivsystem vorhanden wäre. Zudem ist solcher Code häufig von Entwicklern geschrieben, die noch kein gutes Verständnis der Sprache haben und deswegen von schlechter Qualität. Dies könnte die Ergebnisse verfälschen.
- Die Benchmark testet nur die Übersetzung von in sich geschlossenen COBOL Programmen. Alle Testcases sind in sich geschlossen und haben nur Abhängigkeiten auf allfällige Input Dateien. Ein Übliches COBOL Programm ruft aber häufig auch andere Programme mit auf, welches wieder andere Programme aufruft usw. Dies wird hier nicht getestet.
- Nur COBOL Codes, welche mit dem GNUCobol Compiler kompilierbar sind wurden verwendet, es gibt aber noch andere Cobol Dialekte/Versionen, welche damit nicht Kompilierbar sind.
- Command Line Argumente werden nicht unterstützt

5. Fazit und Ausblick

5.1. Weiterführende Arbeit

Es gibt in diesem Bereich noch viel zu Forschen, einige mögliche Ansatzpunkte wären:

- **Systematischer Vergleich von unterschiedlichen Übersetzungstechniken** Es gibt mit LLMs noch viele andere Techniken, welche über einen simplen Prompt hinausgehen, welche getestet werden können. Neben puren LLM Lösungen können aber auch andere Ansätze verfolgt werden, es gibt schon Programme, welche den COBOL Code deterministisch übersetzen können, der produzierte Code ist aber nicht gut. Dieser Code könnte aber evtl. iterativ mit einem LLM verbessert werden.
- **Unterstützung von Command Line Argumenten**
- **Übersetzung von mehreren verknüpften Programmen** Der Grossteil der Programme hat Abhängigkeiten zwischen mehreren Dateien, in der Benchmark wird aber immer nur eine Datei übersetzt, das ist unrealistisch. Die Benchmark sollte so erweitert werden, um Testcases zu ermöglichen, welche das übersetzen von mehreren Dateien erfordert.

- **Datensatz an Industrie-Code** Die Erstellung eines Grossen Datensatzes an COBOL Programmen, welche CODE enthält, welcher repräsentativ für den in der Industrie verwendeten Code ist wäre von grossem Nutzen für die Erstellung und Testing von neuen Tools, sowie das Training eines COBOL-Code-LLMs..

5.2. Zusammenfassung

Im Verlaufe dieses Projektes wurde eine Benchmark entwickelt, welche den objektiven Vergleich von unterschiedlichen COBOL zu Java Übersetzungsmethoden ermöglicht. Dies wird mithilfe von Äquivalenztests und Statischer Code Analyse gemacht. Weiter wurden fünf Übersetzer Vorgestellt und systematisch miteinander verglichen.
Der Code de Benchmark ist frei auf Github verfügbar (Schafhäutle, 2025).

Literaturverzeichnis

Abstract syntax tree [Page Version ID: 1319352066]. (2025, Oktober). Verfügbar 26. November 2025 unter https://en.wikipedia.org/w/index.php?title=Abstract_syntax_tree&oldid=1319352066

akash1295. (2025, September). Cyclomatic Complexity [Section: DSA]. Verfügbar 26. November 2025 unter https://www.geeksforgeeks.org/dsa/cyclomatic-complexity/

Anthropic. (2025, November). Claude Docs. Verfügbar 26. November 2025 unter https://platform.claude.com/docs/en/home

Cosentino, V., Cabot, J., Albert, P., Bauquel, P., & Perronnet, J. (2013). Extracting business rules from COBOL: A model-based framework [ISSN: 2375-5369]. *2013 20th Working Conference on Reverse Engineering (WCRE)*, 409–416. https://doi.org/10.1109/WCRE.2013.6671316

Dau, A. T. V., Dao, H. T., Nguyen, A. T., Tran, H. T., Nguyen, P. X., & Bui, N. D. Q. (2024, August). XMainframe: A Large Language Model for Mainframe Modernization [arXiv:2408.04660 [cs]]. https://doi.org/10.48550/arXiv.2408.04660

Free Software Foundation. (2023, Juli). GnuCOBOL - GNU Project. Verfügbar 23. November 2025 unter https://gnucobol.sourceforge.io/

IBM. (2024, April). IBM watsonx Code Assistant for Z. Verfügbar 23. November 2025 unter https://www.ibm.com/products/watsonx-code-assistant-z

Jaccard-Koeffizient [Page Version ID: 260892731]. (2025, Oktober). Verfügbar 24. November 2025 unter https://de.wikipedia.org/w/index.php?title=Jaccard-Koeffizient&oldid=260892731

Kocetkov, D., Li, R., Ben Allal, L., Li, J., Mou, C., Muñoz Ferrandis, C., Jernite, Y., Mitchell, M., Hughes, S., Wolf, T., Bahdanau, D., von Werra, L., & de Vries, H. (2022). The Stack: 3 TB of permissively licensed source code. *Preprint*.

Kumar, A., Saha, D., Yasue, T., Ono, K., Krishnan, S., Hans, S., Satoh, F., Mitchell, G., & Kumar, S. (2024). Automated Validation of COBOL to Java Transformation. *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 2415–2418. https://doi.org/10.1145/3691620.3695365

Lee, D., Henley, A., Hinshaw, B., & Pandita, R. (2022, Juni). OpenCBS: An Open-Source COBOL Defects Benchmark Suite [arXiv:2206.06260 [cs]]. https://doi.org/10.48550/arXiv.2206.06260

Lei, F., Liu, J., Noei, S., Zou, Y., Truong, D., & Alexander, W. (2025, Juli). Enhancing COBOL Code Explanations: A Multi-Agents Approach Using Large Language Models [arXiv:2507.02182 [cs]]. https://doi.org/10.48550/arXiv.2507.02182

Levenshtein-Distanz [Page Version ID: 259375212]. (2025, September). Verfügbar 24. November 2025 unter https://de.wikipedia.org/w/index.php?title=Levenshtein-Distanz&oldid=259375212

Levy, A., Shalom, B. R., & Chalamish, M. (2024, August). A Guide to Similarity Measures [arXiv:2408.07706 [cs] version: 1]. https://doi.org/10.48550/arXiv.2408.07706

McCabe, T. (1976). A Complexity Measure. *IEEE Transactions on Software Engineering*, *SE-2*(4), 308–320. https://doi.org/10.1109/TSE.1976.233837

Mikejo5000. (2025, Oktober). Code metrics - Maintainability index range and meaning - Visual Studio (Windows). Verfügbar 25. November 2025 unter https://learn.microsoft.com/en-us/visualstudio/code-quality/code-metrics-maintainability-index-range-and-meaning?view=visualstudio

Schafhäutle, D. (2025, November). danielschafi/Cob2Java-Bench [original-date: 2025-10-05T20:08:08Z]. Verfügbar 25. November 2025 unter https://github.com/danielschafi/Cob2Java-Bench

Shah, S., Agarwal, S., Krishnan, S., Kanvar, V., & Chimalakonda, S. (2025). A-COBREX: A Tool for Identifying Business Rules in COBOL Programs [ISSN: 2574-1934]. *2025 IEEE/ACM 47th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*, 5–8. https://doi.org/10.1109/ICSE-Companion66252.2025.00011

Shivani Virmani. (2025, September). Halstead's Software Metrics - Software Engineering [Section: Software Engineering]. Verfügbar 26. November 2025 unter https://www.geeksforgeeks.org/software-engineering/software-engineering-halsteads-software-metrics/

TecnoDigital. (2025, März). Programmiersprache Cobol: Ein vollständiger Überblick. Verfügbar 23. November 2025 unter https://informatecdigital.com/de/COBOL-Programmiersprache%3A-Ein-vollst%C3%A4ndiger-%C3%9Cberblick/

Wu, G., & Martin, E. (2024, März). String Similarity Metrics – Edit Distance — Baeldung on Computer Science. Verfügbar 26. November 2025 unter https://www.baeldung.com/cs/string-similarity-edit-distance

Yan, W., Tian, Y., Li, Y., Chen, Q., & Wang, W. (2023). CodeTransOcean: A Comprehensive Multilingual Benchmark for Code Translation. *arXiv preprint arXiv:2310.04951*.

Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., Zheng, C., Liu, D., Zhou, F., Huang, F., Hu, F., Ge, H., Wei, H., Lin, H., Tang, J., … Qiu, Z. (2025, Mai). Qwen3 Technical Report [arXiv:2505.09388 [cs]]. https://doi.org/10.48550/arXiv.2505.09388

A. Prompts

A.1. System Prompt

```
You are a code conversion assistant.
Your task is to convert COBOL code (which is compilable by the GNUCobol compiler) into

REQUIREMENTS:
- The Java program MUST be equivalent in functionality to the COBOL code.
- The same inputs MUST produce the same outputs.
- The output MUST be ONLY a complete, compilable Java file.
- Do NOT include explanations, comments, or any extra text outside of the code.
- Do NOT wrap the code in markdown fences (```) or language tags.
- The output should ONLY be the CONTENT of the Java file.

IF YOU GENERATE MARKDOWN BRACKETS OR ANY EXTRA TEXT, REMOVE THEM.
```

A.2. Base Prompt

```
BEGIN COBOL CODE:
{cobol_code}
END COBOL CODE
```

A.3. AST Prompt

```
BEGIN COBOL CODE:
{cobol_code}
END COBOL CODE

Additionally here is the Abstract Syntax tree for this cobol programm,
use it to assist in conversion to java:

BEGIN AST:
{ast}
END AST
```